

Contents

Policy: CPI-Tiered Preference Rounds (tiered_rounds)	1
Overview	1
Pipeline	1
Phase 0	1
Preference Rounds	2
Round structure	2
Selection rule within an advisor-round	2
At most one student per advisor per round	2
Tie ordering	2
Termination	2
Manual tie-break UI	2
Audit trace	3
Properties	3
Trade-offs vs other policies	4
When to prefer tiered_rounds	4
Known limitations	5
See also	5

Policy: CPI-Tiered Preference Rounds (tiered_rounds)

Overview

tiered_rounds is an **interactive, round-based allocation policy**. It runs Phase 0 for tier classification (diagnostic only — no tier cap is applied during assignment) and then executes a sequence of preference rounds. In each round n , every still-unassigned student simultaneously offers their n -th preference. Each advisor considers their pool and selects at most one student per round: the highest-CPI candidate wins outright; if two or more students share the top CPI, the operator must make a manual pick before the round advances. The process continues until all students are assigned or a stall is detected.

This policy is available both through the Dash UI and via the CLI. In CLI (batch) mode all ties are resolved automatically by highest CPI (ties in CPI broken by student ID ascending); the manual pick UI is only available in the Dash GUI.

Pipeline

Phase 0 → Preference Rounds ($n = 1, 2, 3, \dots$)

Round 1 is not run separately. There is no class-wise main allocation phase.

Phase 0

Identical to all other policies. Students are tiered by CPI percentile (or quartile if the cohort clusters heavily). N_{tier} windows and faculty max_load values are set. The tier information is **diagnostic**

only — it is displayed in the completion panel's per-tier breakdown but does not restrict which advisors students can be assigned to during the rounds.

Preference Rounds

Round structure

In round n (starting at $n = 1$):

1. Every **unassigned** student offers their n -th preference (i.e., the advisor at position n in their ranked list).
2. Students whose preference list is shorter than n are skipped (only reachable with malformed data; normally every student lists all faculty).
3. Advisors that reached full capacity in a **previous** round are skipped. Advisors that fill up during round n are still available in round n but are skipped from round $n+1$ onwards.

Selection rule within an advisor-round

For each participating advisor, collect all students who offered that advisor in this round. Then:

- **Unique highest CPI** → assign that student immediately. Append a snapshot.
- **CPI tie at the top** → pause the engine. Present a manual pick UI to the operator.

At most one student per advisor per round

Even if an advisor has remaining capacity > 1 , they select at most one student per round. Non-selected students (including those whose preference was ignored because another student won, or who were in a tie and not chosen) advance to round $n+1$ where they offer their $(n+1)$ -th preference.

Tie ordering

When multiple advisors in the same round each have a CPI tie, the ties are presented to the operator one at a time in **decreasing order of the tied CPI value** (highest-stakes tie first).

Termination

- **Complete:** all students assigned. The engine reports the number of rounds used.
 - **Stall:** a full round passes with zero assignments and zero ties — typically caused by malformed data (incomplete preference lists or insufficient total capacity). The engine stops, preserves the partial allocation, and reports the unassigned students.
-

Manual tie-break UI

When a tie is detected:

1. The allocation pauses at that advisor-round.
2. The Preference Rounds panel shows:
 - Round number and advisor name.

- A table of all candidates who offered this advisor this round, with CPI and tier.
 - The tied students highlighted.
 - A dropdown to select exactly one student.
 - A “Confirm selection and continue →” button.
3. After confirmation, the selected student is assigned; all other candidates (tied and non-tied) advance to the next round.
 4. If additional ties exist in the same round, they are presented next in descending tied-CPI order.
 5. Once all ties in the round are resolved, the engine advances to round $n+1$ automatically.

Audit trace

A round-by-round trace is recorded and shown in a collapsible section of the completion panel:

Field	Contents
Round number	Which round this entry describes
Active students	Students who participated in this round
Active preferences	Each student’s n -th preference target
Skipped advisors	Advisors that were full from a previous round
Candidate pools	All students targeting each advisor this round
Unambiguous picks	Immediate assignments (unique highest CPI)
Ties	CPI ties detected, with advisor and tied student IDs
Manual decisions	Operator’s choice for each tie (recorded for reproducibility)
Forwarded	Students moving to the next round
Advisor loads after round	Load snapshot at end of each round

Properties

Property	Behaviour
Merit sensitivity (NPSS)	High — higher-CPI students win their preferred advisor when competing with lower-CPI students for the same advisor in the same round
Preference satisfaction (PSI)	High in balanced cohorts — students advance down their list only if their earlier preferences are taken
Load balance	Variable — each advisor gains at most one student per round, but popular advisors fill faster
Empty-lab guarantee	Indirect — if sufficient total capacity exists and all students list all faculty, everyone is assigned

Property	Behaviour
MSES	High in balanced cohorts — students advance down their list only when earlier preferences are taken, so assignments tend to reflect genuine enthusiasm
Equity Retention Rate	Variable — no load-balancing mechanism; popular advisors fill early, which can concentrate CPI tiers
Transparency / auditability	Highest — every assignment, every tie, and every manual decision is recorded in the trace log
Interactivity	Required in GUI — the operator must resolve all CPI ties manually. Unattended CLI runs are available via <code>--auto-tiebreak</code> .

Trade-offs vs other policies

Aspect	<code>tiered_rounds</code>	<code>least_loaded</code>	<code>cpi_fill</code>
Assignment order	Round-by-preference-rank; CPI breaks ties within a round	Tier-by-tier (arbitrary within tier)	Strict descending CPI
Tier cap	None (N_tier diagnostic only)	Applied per tier; overflow possible via promotion cascade ($A \rightarrow B \rightarrow C$)	None
Competition resolution	Manual tie-break if CPI equal	Automatic (least-load preference)	Automatic (first-come first-served in CPI order)
Load balancing	Implicit (at most 1 per advisor per round)	Explicit (least-load criterion)	Weak (first-preference greedy)
Empty-lab guarantee	Implicit	Indirect	Explicit (Phase 2 design)
Operator involvement	Required for ties in GUI; auto with <code>--auto-tiebreak</code> in CLI	Optional override	Optional override

When to prefer `tiered_rounds`

- When **full auditability** is a requirement — every decision, including tie-breaks, is on record.
- When the allocation coordinator wants to **personally resolve CPI ties** rather than delegating that decision to an algorithm.

- In **small or well-curated cohorts** where ties are rare and the coordinator knows the candidates.

Known limitations

- **GUI requires manual tie resolution** — CPI ties halt the Dash UI until the operator makes a pick. For unattended batch runs, use `--auto-tiebreak` (CLI), which resolves all ties by highest CPI automatically.
 - **No explicit load balancing** — an advisor that receives a student in round 1 may receive another in round 2 if they still has capacity, potentially diverging from load-balanced outcomes.
 - **Overflow count is structural, not anomalous** — because `N_tier` windows are never applied during assignment, students may be placed at a rank $> N_{\text{tier}}$ whenever their earlier preferences are taken. The completion panel shows this as a **blue informational badge** (“outside N-tier window — expected for tiered rounds”) rather than a red error. The NPSS score is not affected.
-

See also

- `tiered_ll` — hybrid variant: runs tiered rounds 1..k (same tie-break mechanics), then switches to an automatic LL-HP backfill; adds an empty-lab guarantee.